Chapter 7

Shared-Memory Parallelism with OpenMP

7.1 Learning Objectives and Scope

OpenMP lets us add parallel execution to a C program a step at a time. We begin with independent array elements, then combine results, reuse a team across several loops, and finally express work as tasks. At each step, ask: who writes each result, and when may another thread read it?

After this chapter, students should be able to compile an OpenMP C program; explain a parallel region, worksharing construct, and implicit barrier; classify variables by data scope; use a reduction; choose a loop schedule; parallelize a nested loop; decide when `nowait` is safe; and create independent or dependent tasks. Chapter 5 explained shared-state correctness and Chapter 6 implemented explicit POSIX threads. Here we express shared-memory work through OpenMP directives. Chapter 8 studies the algorithms independently of any programming interface; Chapter 10 develops measurement methodology.

7.2 Install and Verify OpenMP

Follow the installation subsection for your operating system, run the verification program, and then continue with the fork–join model. You do not need to complete the other platforms' instructions.

The Chapter 7 examples are C11 programs. Build them from the *project root*, which contains the `examples/chapter7` directory. Use a GCC build with OpenMP support and pass `-fopenmp` while compiling and linking. GCC's OpenMP guide explains that this flag enables directives and links the runtime. The small `check_openmp.c` program requires the `_OPENMP` macro, starts a team, and compares the runtime team size with a reduction count.

7.2.1 macOS: Homebrew GCC

Install Homebrew using its official instructions if it is not already installed. In Terminal, run the following commands from the project root. The Homebrew GCC formula currently provides `gcc-16`; if the versioned executable changes, use the name printed by Homebrew instead. Use this compiler rather than Apple's `cc` for these commands.

Listing 7.1: macOS: install GCC and verify OpenMP

```
brew install gcc
gcc-16 --version
gcc-16 -std=c11 -O2 -Wall -Wextra -Werror -fopenmp \
  examples/chapter7/check_openmp.c -o check_openmp
OMP_NUM_THREADS=4 ./check_openmp
```

7.2.2 Linux: Ubuntu or Debian

Install GCC through the distribution package manager. For Ubuntu or Debian, use the following commands in a terminal. Ubuntu's GCC setup guide documents its compiler package. On another Linux distribution, install its GCC package and use the same `gcc -fopenmp` build command.

Listing 7.2: Ubuntu or Debian: install GCC and verify OpenMP

```
sudo apt update
sudo apt install gcc
gcc --version
gcc -std=c11 -O2 -Wall -Wextra -Werror -fopenmp \
  examples/chapter7/check_openmp.c -o check_openmp
OMP_NUM_THREADS=4 ./check_openmp
```

7.2.3 Windows: native MSYS2 UCRT64

Install MSYS2, then open its *UCRT64* terminal. The ordinary MSYS terminal uses a different environment. Update packages first; if asked to close the terminal, reopen UCRT64 and run the update again. The MSYS2 environment guide recommends UCRT64, and its libgomp package supplies the GCC OpenMP runtime.

Listing 7.3: Windows: install UCRT64 GCC and verify OpenMP

```
pacman -Suy
pacman -S mingw-w64-ucrt-x86_64-gcc \
  mingw-w64-ucrt-x86_64-libgomp
gcc --version
gcc -std=c11 -O2 -Wall -Wextra -Werror -fopenmp \
  examples/chapter7/check_openmp.c -o check_openmp.exe
OMP_NUM_THREADS=4 ./check_openmp.exe
```

7.2.4 Windows alternative: Ubuntu under WSL

Students using Linux tools throughout the course may instead open PowerShell as Administrator and run `wsl --install -d Ubuntu`, then restart if prompted. Complete Ubuntu's first-run account setup, open its terminal, and follow the Linux commands above *inside Ubuntu*. The executable produced there runs inside Ubuntu, not as a native Windows program. See Microsoft's WSL installation guide.

7.2.5 Read the verification result

A typical four-thread run prints `team=4 counted=4`; a runtime may use fewer threads if constrained. The two numbers must agree. If compilation reports that `omp.h` is missing or `_OPENMP` is disabled, check the selected compiler and `-fopenmp`. On macOS, `cc` may select Apple Clang instead of Homebrew GCC. On Windows, confirm the terminal says *UCRT64*; if a program cannot find `libgomp-1.dll`, run it in that terminal. If the source path is missing, change to the project root before compiling. Once this check succeeds, build the remaining examples with the

same compiler and flag; use `gcc` on Linux or MSYS2 and the installed versioned `gcc` name on macOS.

7.3 The Fork–Join Model

OpenMP adds directives and runtime functions to C, C++, and Fortran. The initial thread encounters a `parallel` region and forms a team. At the end of the region the team synchronizes, and only the initial thread proceeds. A `parallel for` combines team creation with distributing loop iterations. The compiler must be told to enable OpenMP; otherwise a C compiler may ignore the pragmas and still produce a serial program.

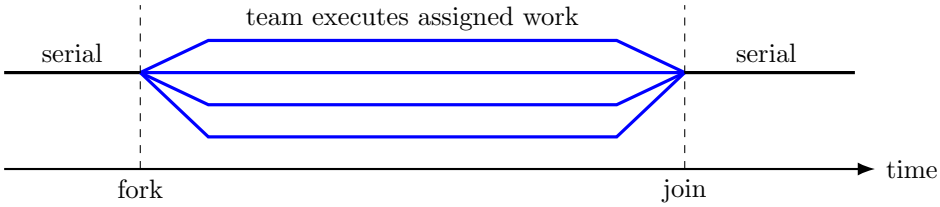


Figure 7.1: A parallel region creates a team and joins it before subsequent serial work. Lines indicate possible concurrent execution, not measured time.

A useful distinction is that `parallel` creates a team, while `for` distributes loop iterations among a team that already exists. With `parallel for`, these operations appear together. Each iteration executes once; each thread does not execute the whole loop. Later, we will put several `for` constructs inside one `parallel` region to reuse the team.

7.4 A Complete Loop and Reduction Example

Our first goal is to double every array element and then sum the original array. These operations need different treatments: doubling writes separate destinations, whereas summing combines contributions into one result.

The file `examples/chapter7/openmp_examples.c` contains three checked examples. Read the scaling and sum loops first; the prime-count tasks in the same file are explained later. From the project root, using the compiler established above, build and run:

Listing 7.4: Build and run the Chapter 7 examples

```
gcc-16 -std=c11 -O2 -Wall -Wextra -Werror -fopenmp \
  examples/chapter7/openmp_examples.c -o openmp_examples
OMP_NUM_THREADS=4 ./openmp_examples
```

Substitute `gcc` for `gcc-16` on Linux or MSYS2, and use a `.exe` output name in MSYS2. The program checks every scaled element, compares its sum with $1003(1004)/2 = 503{,}506$, and checks eight prime-count task outputs against known values. A successful run prints:

```
scaled=1003 sum=503506 expected=503506 tasks=8
```

`OMP_NUM_THREADS` is a request to the runtime; the actual team size may depend on runtime limits and nesting.

Listing 7.5: Independent iterations and a reduction from the runnable example

```
#pragma omp parallel for default(none) shared(a, b, n) schedule(static)
for (int i = 0; i < n; ++i)
    b[i] = 2 * a[i];

long long total = 0;
#pragma omp parallel for default(none) shared(a, n) reduction(+:total)
for (int i = 0; i < n; ++i)
    total += a[i];
```

Each scaling iteration writes a unique `b[i]` and only reads `a[i]`; no element is updated by two iterations. In the second loop, `reduction(+:total)` gives each participating thread a private partial initialized to the additive identity, then combines partials when the loop finishes. An ordinary shared `total += a[i]` would race. An integer `long long` result is exact here because the chosen input sum fits that type. Floating-point reductions can produce different rounding from a serial left-to-right sum.

7.4.1 Trace a small input

For a hand calculation, replace the large input mentally with $a = [1, 2, 3, 4]$. The scaling loop produces $b = [2, 4, 6, 8]$, regardless of which thread handles each index. For the reduction, suppose two threads receive two elements each:

Table 7.1: One possible two-thread execution of the sum

Thread	Assigned input values	Private partial sum
0	$1, 2$	$0 + 1 + 2 = 3$
1	$3, 4$	$0 + 3 + 4 = 7$

At the end, OpenMP combines the partials with the original zero value of `total`: $0+3+7 = 10$. The assignment shown is illustrative; correctness must hold for any allowed assignment. The barrier at the end of the combined construct ensures that serial code after it sees the finished result.

Read the directive from left to right: `parallel for` distributes the loop across a team; `default(none)` asks us to state data scope; `shared(a,n)` makes the input and bound available to the team; and `reduction(+:total)` defines how contributions are combined. The next section explains these scope choices.

7.4.2 Other built-in C/C++ reductions

A reduction specifies how to combine private contributions. It does not replace the calculation in the loop body. The following table covers the built-in C/C++ identifiers in OpenMP 5.2, Section 5.5.3, including the deprecated subtraction identifier.

Table 7.2: Built-in reductions: private starting values and combination rules

Identifier	Private initial value	Combine using
`+`	0	Addition
`-` (deprecated)	0	Addition, not subtraction
`*`	1	Multiplication
`min`	Largest representable value	Minimum
`max`	Smallest representable value	Maximum
`&`	All bits set	Bitwise AND
`|`	0	Bitwise OR
`^`	0	Bitwise XOR
`&&`	1	Logical AND
`||`	0	Logical OR

These initializers apply to private copies. Initialize the original variable too: its starting value participates in the final combination. In particular, use 1 for a product and appropriate limits for extrema. Initializing a maximum to zero gives a wrong answer for an array containing only negative values. Bitwise operators require integral operands; logical operators combine truth values. There is no built-in division reduction.

7.4.3 One loop, several results

The runnable file `examples/chapter7/reduction_variants.c` uses the input $[1, 2, 3, 4]$. It initializes `sum=0`, `product=1`, `low=INT_MAX`, and `high=INT_MIN`. Logical accumulators start at 1 for "all" and 0 for "any". Unsigned bitwise accumulators start at `~0u` for AND and zero for OR and XOR. A balance starts at 100.

Listing 7.6: Computing several reductions in the same loop

```
#pragma omp parallel for default(none) shared(a) \
    reduction(+:sum,balance) reduction(*:product) \
    reduction(min:low) reduction(max:high) \
    reduction(&&:all_positive) reduction(||:any_even) \
    reduction(&:bits_and) reduction(|:bits_or) reduction(^:bits_xor)
for (int i = 0; i < 4; ++i) {
    sum += a[i];
    balance -= a[i];
    product *= a[i];
    if (a[i] < low) low = a[i];
    if (a[i] > high) high = a[i];
    all_positive = all_positive && (a[i] > 0);
    any_even = any_even || (a[i] % 2 == 0);
    bits_and &= (unsigned)a[i];
    bits_or |= (unsigned)a[i];
    bits_xor ^= (unsigned)a[i];
}
```

For this input, multiplication gives $1 \times 2 \times 3 \times 4 = 24$, the minimum is 1, and the maximum is 4. Every element is positive, and at least one is even, so both logical results are 1. In three-bit notation, the values are 001, 010, 011, and 100. Their AND is 000, OR is 111, and XOR is 100: decimal results 0, 7, and 4. Bitwise AND examines individual bits; logical AND asks whether every tested condition is true.

Listing 7.7: Build and run the reduction variants

```
gcc-16 -std=c11 -O2 -Wall -Wextra -Werror -fopenmp \
  examples/chapter7/reduction_variants.c -o reduction_variants
OMP_NUM_THREADS=4 ./reduction_variants
```

Use `gcc` on Linux or MSYS2, as in the installation section. The program checks every result and prints:

```
sum=10 product=24 min=1 max=4 balance=90
all_positive=1 any_even=1 and=0 or=7 xor=4
```

7.4.4 Why subtraction uses an addition combiner

Although `reduction(-:x)` exists in OpenMP 5.2, it is deprecated and combines partials by addition. Prefer `reduction(+:balance)` with `balance -= a[i]`, as above. For two threads processing two values each, private balances can become -3 and -7. They combine with the original balance to give $100 + (-3) + (-7) = 90$. The private balances start at zero, not 100. This computes the original balance minus the sum of contributions; it does not subtract partial results from one another.

Use a type large enough for the sum or product: a reduction does not prevent integer overflow. Floating-point results can vary with combination order. These examples use small integers to make every expected value easy to check.

7.5 Data Scope and Lifetime

`default(none)` requires explicit scoping for variables whose data-sharing attributes are not predetermined, making accidental sharing easier to find. A loop's canonical induction variable is predetermined private. This table describes the scope of variables in a region, not the memory model introduced in Chapter 5.

Table 7.3: OpenMP data-sharing choices for the Chapter 7 examples

Clause	Meaning	Appropriate use
`shared(x)`	Team accesses one object	Read-only input array; disjoint output elements.
`private(x)`	Each thread has a new, uninitialized object	Scratch storage assigned before use.
`firstprivate(x)`	Each task/thread starts with a copy	Capture one loop iteration's task argument.
`reduction(+:x)`	Private partials combined with addition	Scalar sum whose operator is valid for reordering.

Scope alone does not prove correctness. `shared(b)` is safe for the scaling loop because iteration i exclusively owns element b_i. The task example below captures a fresh value for each task and writes a different output slot. The shared arrays and captured values remain valid until the enclosing parallel region completes.

For example, `private(x)` does not copy an existing value of `x`. If each thread needs to start with the value 5, `firstprivate(x)` provides that initial copy. Changing a private copy does not update the original variable. For a result that must combine contributions, use a suitable reduction instead.

7.6 Scheduling Independent Iterations

A schedule decides which thread receives which iterations; it does not make dependent iterations independent. For a loop of $N = 10$ iterations and $P = 3$ threads, `schedule(static,2)` creates chunks $[0, 2)$, $[2, 4)$, $[4, 6)$, $[6, 8)$, $[8, 10)$ and assigns chunks cyclically to threads. Numbering threads 0, 1, and 2, thread 0 receives indices $0, 1, 6, 7$; thread 1 receives $2, 3, 8, 9$; and thread 2 receives $4, 5$. Thus their iteration counts are 4, 4, and 2. Assignment order does not imply completion order: one thread can finish its chunks before another. Here $[0, 2)$ means indices 0 and 1; the upper endpoint is excluded.

Table 7.4: Choosing a schedule after proving loop independence

Schedule	Good fit	Cost or caution
`static`	Similar iteration costs; contiguous access	Poor balance if expensive iterations cluster.
`dynamic,chunk`	Unpredictable iteration costs	More assignment overhead; small chunks can hurt locality.
`guided,chunk`	Large irregular loop	Chunk sizes decrease, so early large chunks can still be uneven.
`runtime`	Experiment without recompiling	Set `OMP_SCHEDULE`; results depend on that setting.

7.7 Complete Matrix-Vector and Nested-Loop Program

The next program has two distinct worksharing loops. The first uses `collapse(2)` to distribute the 4×3 elementwise scaling iterations as one logical iteration space. The second assigns one row of a matrix-vector product to each loop iteration. Its local `sum` is private because it is declared inside the loop body; only iteration i writes `y[i]`. There is no cross-row accumulation and no reduction clause is needed.

Listing 7.8: Build and run the matrix-vector example

```
gcc-16 -std=c11 -O2 -Wall -Wextra -Werror -fopenmp \
  examples/chapter7/matrix_vector.c -o matrix_vector
OMP_NUM_THREADS=4 ./matrix_vector
```

The expected output is `matvec=[14,32,50,68] doubled=ok`. The first result is $1(1) + 2(2) + 3(3) = 14$; the other three rows are checked against separately stated expected values. The program checks every scaled matrix element too. The core OpenMP code is shown below; the named source file also includes inputs, checks, and a runnable `main`.

Listing 7.9: Collapsed elementwise loop and row-owned matrix-vector result

```
/* The collapsed iterations still own distinct matrix elements. */
#pragma omp parallel for collapse(2) default(none) shared(a, doubled)
for (int i = 0; i < ROWS; ++i)
    for (int j = 0; j < COLS; ++j)
        doubled[i][j] = 2 * a[i][j];

/* Row i is the only writer of y[i]; sum is private to that iteration. */
#pragma omp parallel for default(none) shared(a, x, y)
for (int i = 0; i < ROWS; ++i) {
    int sum = 0;
    for (int j = 0; j < COLS; ++j)
        sum += a[i][j] * x[j];
    y[i] = sum;
}
```

To follow the matrix-vector loop, focus on row 0. Its local `sum` starts at zero, then becomes 1, 5, and 14 as the inner loop visits the three columns. It finally stores 14 in `y[0]`. Another iteration can compute row 1 at the same time using its own `sum`, producing $4(1) + 5(2) + 6(3) = 32$. The inner loop remains serial within each row; parallelism comes from processing different rows.

For the scaling loop, `collapse(2)` makes the pairs $(0,0), (0,1), (0,2), (1,0), \ldots, (3,2)$ available for distribution. It exposes twelve independent element updates instead of only four outer-loop iterations.

`collapse(2)` is valid here because both loops are canonical, rectangular, and independent: computing `doubled[i][j]` never reads another result element. It does not authorize collapsing the inner dot-product loop into independent writes to `y[i]`; those writes would conflict. For large matrices, the choice between row ownership and finer-grained work should be justified by the algorithm's dependencies and the measured workload.

7.8 Synchronization and Barriers

The end of a worksharing `for` normally has an implicit barrier. A second loop that reads the entire first loop's output must wait for it. `nowait` removes that barrier and is safe only when no subsequent operation needs work that another thread may still be doing. A `single` region executes once per team and has an implicit barrier at its end unless `nowait` is present. `atomic` protects a supported single-location update; `critical` serializes a block. A reduction is usually the clearer expression for combining loop values. The choice of mechanism follows the shared-state invariant from Chapter 5.

7.8.1 A complete three-phase worksharing program

The file `examples/chapter7/worksharing_phases.c` has three loops in one parallel region. Phase A writes `a`; phase B writes `b` without reading `a`. It is therefore safe for a thread that finished its A iterations to start B while another thread is still doing A. Phase C reads `a[i]` and `b[N-1-i]`. A thread other than the one running C may own that `b` element, so all A and B iterations must finish before any C iteration starts.

Listing 7.10: Build and run the worksharing phases

```
gcc-16 -std=c11 -O2 -Wall -Wextra -Werror -fopenmp \
  examples/chapter7/worksharing_phases.c -o worksharing_phases
OMP_NUM_THREADS=4 ./worksharing_phases
```

The program checks all 16 results and prints `verified=16 last=18`. For $N = 16$, $c_i = (i + 1) + 2(N - i) = 2N + 1 - i$, so the last value is 18. The complete source includes initialization, checks, and a runnable `main`; the three worksharing loops are:

Listing 7.11: Safe nowait followed by a necessary implicit barrier

```
#pragma omp parallel default(none) shared(a, b, c)
{
    /* Phase A and phase B do not read each other's outputs. */
    #pragma omp for nowait schedule(static)
    for (int i = 0; i < N; ++i)
        a[i] = i + 1;

    /* The implicit barrier at this loop's end finishes A and B. */
    #pragma omp for schedule(static)
    for (int i = 0; i < N; ++i)
        b[i] = 2 * (i + 1);

    /* b[N-1-i] may have been written by a different thread. */
    #pragma omp for schedule(static)
    for (int i = 0; i < N; ++i)
        c[i] = a[i] + b[N - 1 - i];
}
```

The `nowait` on A permits overlap because B is independent. The end of B has an implicit barrier. Every thread must have finished its assigned A and B iterations before any thread passes that barrier, so C can read both arrays. Adding `nowait` to B would remove that guarantee and could let C read another thread's unfinished `b` element. A successful run alone cannot prove that such a change is safe: an unsafe schedule may happen to produce the expected values.

Consider $N = 4$ with two threads and contiguous static chunks. Thread 0 computes indices 0 and 1; thread 1 computes 2 and 3. Phase C at index 0 needs $a[0] = 1$ and $b[3] = 8$, so its result is 9. Thread 0's completion of its own B iterations says nothing about whether thread 1 has written `b[3]`. The barrier supplies exactly that missing guarantee.

7.9 Tasks for Irregular Work

A `task` records work that a team thread may execute later. `single` creates the tasks once; the surrounding `parallel` region provides a team to run them. In the runnable example, task i counts primes up to $5000(i + 1)$, so task costs differ. Each task receives its own copy of `slot` and writes `out[slot]`. `taskwait` waits for the child tasks created by the thread executing this `single` block. The program checks the results after the parallel region.

Listing 7.12: Task creation with a stable per-task argument

```
#pragma omp parallel default(none) shared(out)
{
    #pragma omp single
    {
        for (int i = 0; i < TASKS; ++i) {
            int slot = i;
            #pragma omp task default(none) firstprivate(slot) shared(out)
            out[slot] = count_primes(5000 * (slot + 1));
        }
        #pragma omp taskwait
    }
}
```

Follow the first two iterations of the task-creation loop. The first task captures `slot=0` and will write `out[0]`; the second captures `slot=1` and will write `out[1]`. Either may start first. The creating thread can advance the loop while an earlier task is still pending, which is why each task needs a stable captured value. `single` chooses one thread to create the tasks, but does not restrict execution of those tasks to that thread.

A task is a unit of work, not a newly created thread. The existing team executes the tasks. In this example, `taskwait` marks where the creating task waits for its child tasks; the enclosing region's synchronization also ensures completion before the serial validation code runs.

The task graph here has eight independent nodes with different costs. The example exposes task capture and ownership without claiming a speedup. For such small tasks, creation overhead can outweigh useful parallel work. Tasks become more useful when each unit is substantial and the work is irregular, such as recursively generated subproblems. A cutoff can execute tiny subproblems directly. Dependency clauses can impose ordering between tasks that share data; independent output slots need none.

7.10 Complete Task-Dependency Program

OpenMP's `depend` clause expresses a producer–consumer edge through a storage location. In this example, two tasks may compute `left` and `right` concurrently. A third task declares both as inputs, so it cannot read either value until the corresponding producer task has completed. All three tasks are created by the same `single` task inside one team. A `taskgroup` waits for the group to finish before the enclosing region ends.

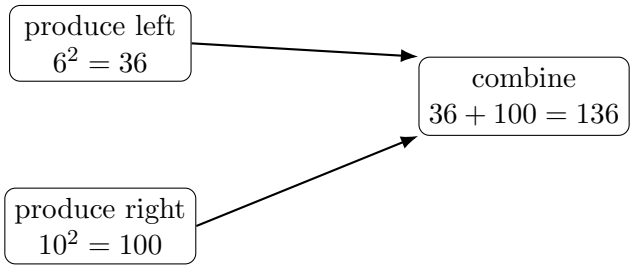


Figure 7.2: The OpenMP dependency clauses permit the two producers to overlap and make the combine task wait for both.

Listing 7.13: Build and run the dependent-task example

```
gcc-16 -std=c11 -O2 -Wall -Wextra -Werror -fopenmp \
  examples/chapter7/task_dependencies.c -o task_dependencies
OMP_NUM_THREADS=4 ./task_dependencies
```

The checked output is `left=36 right=100 total=136`. The producers are created before the consumer so its dependency clauses can match those earlier sibling tasks. Creation order alone would not make the consumer wait for their completion. The complete source file includes the final validation and printed result.

Listing 7.14: Producers and consumer linked by OpenMP dependencies

```
int left = 0, right = 0, total = 0;

#pragma omp parallel default(none) shared(left, right, total)
{
    #pragma omp single
    {
        #pragma omp taskgroup
        {
            #pragma omp task default(none) shared(left) depend(out: left)
            left = 6 * 6;

            #pragma omp task default(none) shared(right) depend(out: right)
            right = 10 * 10;

            /* This task waits for both producers, regardless of order. */
            #pragma omp task default(none) shared(left, right, total) \
                             depend(in: left, right) depend(out: total)
            total = left + right;
        }
    }
}
```

The `depend` list items must identify the same storage locations in producer and consumer tasks. `depend(out: left)` publishes one producer result; `depend(in: left, right)` waits for both preceding outputs. The final read in `main` happens after the parallel region, when all team work has completed. This example assumes the OpenMP task-dependency feature supported by current OpenMP-capable GCC builds.

7.11 Common OpenMP Errors

- **Ignored directive:** build without an OpenMP flag and mistakenly treat serial output as a parallel test. Check the compiler command and, when relevant, `omp_get_num_threads()` inside a region.
- **Shared accumulator:** update one scalar from multiple iterations without a reduction or other synchronization.
- **Premature consumption:** add `nowait` before code that requires all output elements.
- **Task capture:** give deferred tasks one changing loop variable instead of a private copy.
- **Dependent loop:** parallelize an iteration that reads a prior iteration's output. Scheduling cannot repair the dependency.

7.12 Summary

OpenMP lets a compiler and runtime manage teams, loop assignment, reductions, and tasks. A collapsed loop still needs independent iterations; a task dependency identifies the producer that must finish before a consumer reads. Correct programs need an ownership argument for every write, explicit scope choices, and barriers or dependencies where later work needs earlier results.

7.13 Exercises

1. Change the matrix-vector example to a 5×3 matrix. Update the expected results and state the owner of every output element.

2. For ten equal-cost iterations on three threads, list the chunks produced by a static schedule with chunk size 2. Explain why a small dynamic chunk is unlikely to help.

3. Modify the sum to count values exceeding 500 with a reduction. Give the expected count and explain how you checked it.

4. In the worksharing-phases program, explain why A may use `nowait` but B may not. Identify one C read that can depend on another thread's B iteration.

5. Change the prime-count limits so later tasks are much larger. Explain what data is captured, who owns each output, and why `taskwait` is needed. Update the expected results before running the program.

6. Add a fourth task to the dependency example that doubles `total`. Give it an appropriate `depend` clause and explain when `main` may read the final value.

7.14 Selected Worked Answers

For Exercise 2, the five chunks are $[0, 2)$, $[2, 4)$, $[4, 6)$, $[6, 8)$, and $[8, 10)$. Threads receive chunks in round-robin order, so threads 0 and 1 each get two chunks and thread 2 gets one chunk. Equal-cost iterations give little imbalance to correct; dynamic assignment adds overhead. For Exercise 3, integers 501 through 1003 are counted, giving $1003 - 501 + 1 = 503$. For Exercise 4, C iteration $i = 0$ reads `b[15]`, which may belong to another thread's B chunk. Removing B's barrier can let that read happen before the write. A and B themselves have no read-after-write dependency.